

05-程序分析与符号执行复习讲义

目录

1	05 程序分析与符号执行复习讲义
---	------------------

2

1 05 程序分析与符号执行复习讲义

对应资料:

- Slides/Slides/Ch9-1 Dataflow Analysis.pdf
- Slides/Slides/Ch9-2-Symbolic Execution.pdf
- Slides/Slides/Ch12-1-Interprocedure Analysis.pdf
- Slides/Slides/Ch12-2 Pointer Analysis.pdf
- Review/compile.pdf
- Exam/2025.pdf 符号执行与指针别名题

1.1 一、数据流分析框架

数据流分析在 CFG 上计算程序点的事实。

基本元素:

<code>facts</code>	: 数据流事实集合
<code>in[B], out[B]</code>	: 基本块入口和出口事实
<code>transfer function</code>	: 基本块内部如何改变事实
<code>meet function</code>	: 多个前驱或后继事实如何合并

前向分析:

```
in[B] = meet(out[P]) for P in pred(B)
out[B] = transfer_B(in[B])
```

后向分析:

```
out[B] = meet(in[S]) for S in succ(B)
in[B] = transfer_B(out[B])
```

1.2 二、Worklist 算法

朴素迭代每次更新全部基本块。Worklist 只在依赖事实变化时更新相关块。

前向算法:

```

初始化所有 in/out
把入口或全部基本块加入 worklist
while worklist 非空:
    取出 B
    old = out[B]
    in[B] = meet(out[P]) for P in pred(B)
    out[B] = transfer_B(in[B])
    if out[B] != old:
        把 succ(B) 加入 worklist

```

1.3 三、三种经典数据流分析

1.3.1 可达定值分析

问题：哪些定义能不被覆盖地到达当前位置。

```

方向   : 前向
meet   : union
初值   : emptyset
形式   : out[B] = gen[B] union (in[B] - kill[B])

```

1.3.2 可用表达式分析

问题：哪些表达式在到达当前位置前一定计算过，并且操作数没有被重新定义。

```

方向   : 前向
meet   : intersection
初值   : 全集; 入口 out 初始化为空集
形式   : out[B] = gen[B] union (in[B] - kill[B])

```

1.3.3 活跃变量分析

问题：哪些变量的当前值在之后还会被使用。

```

方向   : 后向
meet   : union
初值   : emptyset
形式   : in[B] = use[B] union (out[B] - def[B])

```

1.4 四、敏感性

```

flow-sensitive      : 区分程序点和语句顺序
path-sensitive      : 区分不同控制流路径
context-sensitive   : 区分函数调用上下文
field-sensitive     : 区分对象字段

```

答题时写清：

- 精度越高，状态数和计算成本越高。
- 符号执行提供路径敏感性。
- 指针分析常讨论流敏感、上下文敏感、字段敏感。

1.5 五、符号执行

符号执行为输入变量分配符号值，沿路径执行并维护路径条件。

过程：

1. 输入变量变为符号，例如 `x = alpha_x`。
2. 遇到赋值，更新符号状态。
3. 遇到分支，路径分裂。
4. 每条路径维护 path condition。
5. 使用约束求解器判断路径可满足。

优点：

- 路径敏感，能发现深层错误。
- 可生成测试输入。

成本：

- 路径爆炸。
- 循环和递归处理复杂。
- 外部函数建模困难。

缓解方法：

- State Merging。
- Loop Induction。
- Concolic Execution。

1.6 六、SAT、CNF 与 Tseitin

CNF:

$(l_{11} \text{ or } l_{12} \text{ or } \dots) \text{ and } (l_{21} \text{ or } l_{22} \text{ or } \dots) \text{ and } \dots$

可满足性:

存在一组布尔变量赋值, 使公式为真。

同时可满足:

F_1 可满足, 当且仅当 F_2 可满足。

Tseitin 变换:

1. 为公式语法树中每个子表达式引入新变量。
2. 用 CNF 约束新变量与子表达式语义等价。
3. 加入根节点变量为真的约束。
4. 生成与原公式同时可满足的 CNF, 长度线性增长。

常用等价转 CNF:

$x \leftrightarrow (a \text{ and } b)$
 $(\text{not } x \text{ or } a) \text{ and } (\text{not } x \text{ or } b) \text{ and } (x \text{ or not } a \text{ or not } b)$

$x \leftrightarrow (a \text{ or } b)$
 $(x \text{ or not } a) \text{ and } (x \text{ or not } b) \text{ and } (\text{not } x \text{ or } a \text{ or } b)$

1.7 七、DPLL 与 CDCL

DPLL:

- 选择一个变量赋值。
- 单子句传播。
- 冲突则回溯。
- 遍历搜索空间, 直到找到解或证明不可满足。

CDCL:

- 在 DPLL 基础上进行冲突分析。
- 学习冲突子句。
- 非时序回溯。
- 实际 SAT 求解器主流方法。

1.8 八、指针别名分析

别名：两个表达式表示同一个内存位置。

May-alias:

- 分析结果为 no，表示两者一定不别名。
- 分析结果为 yes，表示两者存在别名关系的上近似。

Must-alias:

- 分析两个表达式是否一定指向同一内存位置。

1.9 九、Andersen 指针分析

Andersen 是 inclusion-based 的 May 指针分析。

约束:

```

y = &x      : x in pts(y)
y = x       : pts(x) subset pts(y)
*y = x      : for v in pts(y), pts(x) subset pts(v)
y = *x      : for v in pts(x), pts(v) subset pts(y)

```

图闭包步骤:

1. $y = \&x$ 加入 points-to 边 $y \rightarrow x$ 。
2. $y = x$ 加入指针流边 $x \Rightarrow y$ 。
3. 若 $*y = x$ ，对 $\text{pts}(y)$ 中每个 v 加边 $x \Rightarrow v$ 。
4. 若 $y = *x$ ，对 $\text{pts}(x)$ 中每个 v 加边 $v \Rightarrow y$ 。
5. 新 points-to 事实产生后，重新触发相关 load/store 约束。
6. 迭代到不动点。

1.10 十、Steensgaard 指针分析

Steensgaard 是 unification-based 分析，把包含关系改成相等关系。

```

y = &x      : x in pts(y)
y = x       : pts(x) = pts(y)
*y = x      : for v in pts(y), pts(x) = pts(v)
y = *x      : for v in pts(x), pts(v) = pts(y)

```

特点：

- 效率高，可用并查集实现。
- 精度低于 Andersen。

1.11 十一、Datalog

Datalog 由 facts 和 rules 组成。

规则形式：

$$P(a,b,\dots) :- A(\dots), B(\dots), \dots$$

可达定值示例：

```

def(B, N, X)           : B 的第 N 条语句定义变量 X
succ(B, N, C)          : C 是 B 的后继
rd(B,N,C,M,X)         : C.M 对 X 的定义到达 B.N

rd(B,N,B,N,X) :- def(B,N,X)
rd(B,N,C,M,X) :- rd(B,N-1,C,M,X), def(B,N,Y), X != Y
rd(B,0,C,M,X) :- rd(D,N,C,M,X), succ(D,N,B)

```

1.12 十二、自测题与答案

1.12.1 题 1：可达定值、可用表达式、活跃变量的方向和 meet。

答案：

```

可达定值：前向，union
可用表达式：前向，intersection
活跃变量：后向，union

```

1.12.2 题 2：符号执行的核心成本是什么？

答案：路径爆炸。

1.12.3 题 3：Andersen 与 Steensgaard 的区别。

答案：Andersen 使用包含约束，精度高、成本高；Steensgaard 使用相等约束，可用并查集实现，效率高、精度低。